WORKING PAPER — SYSTEMS ANALYSIS

Submitted for Comment | Not Peer Reviewed

The Two-Layer Efficiency Stack:

FlashAttention (Operation Efficiency) and Power Metric (Allocation Efficiency) as Independent, Compounding Layers of the Inference Compute Stack

Cole Cantrell

Independent Researcher | cole@paradigmbridge.tech

paradigmbridge.tech

April 2026

Abstract

This paper is a combination calculator, not a new algorithm. It quantifies the compound efficiency of two fully independent methods that address different layers of the inference compute stack and can be deployed simultaneously without modification to either. FlashAttention (Dao et al., 2022/2023) is an exact attention algorithm that reorganizes memory access to reduce bandwidth consumption per operation by 15-78% depending on sequence length. It does not prune tokens, skip computation, or change what is computed — only how. The power metric framework (Cantrell 2026) is an allocation signal that operates at a completely different level: it identifies unproductive training runs or inference samples and stops them early, saving 21-43% of training compute (hypothesized, Paper 1) or 92.7% of sampling compute (simulation, Paper 2). These two methods address distinct bottlenecks (memory bandwidth vs. allocation waste) and compound multiplicatively.

At sequence length 4,096 tokens, combined inference savings reach 97.4% (39.1x multiplier), combining FlashAttention (65%) with adaptive sampling reduction (92.7%, Paper 2 simulation). Combined training savings reach 75.2% (4.0x), using the hypothesized 29% training reduction from Paper 1 signal analysis. At sequence length 16,384 tokens, combined inference savings reach 98.4% (62.3x). These results are computed from published FlashAttention benchmarks (Dao et al., 2022, 2023) and power metric results from prior work (Cantrell 2026). No new experiments are required: the combination formula is total_savings = 1 - (1 - FA_savings) × (1 - PM_savings), which follows from the independence of the two optimizations. We propose that FlashAttention and the power metric form a natural two-layer efficiency stack and suggest the Intelligence Per Watt metric (Mirhoseini et al., 2025) as the unified measure of the combined improvement.

Keywords: FlashAttention, memory bandwidth, IO-awareness, power metric, compute efficiency, two-layer stack, multiplicative savings, intelligence per watt, training efficiency

---

1. Introduction

Concrete example: a model training run with FlashAttention enabled achieves 65% memory bandwidth savings per attention operation at sequence length 4K. The power metric simultaneously reduces sampling compute by 92.7% (inference) or hypothesizes 29% training reduction. These savings do not cancel each other — FlashAttention makes each step cheaper, the power metric reduces how many steps happen. The combined multiplier is their product. The cost of training and deploying large language models has motivated two largely independent lines of research: hardware efficiency and allocation efficiency. Hardware efficiency research asks: given that a compute operation must happen, how can we execute it faster? Allocation efficiency research asks: given a compute budget, which operations are worth doing at all?

FlashAttention (Dao et al., 2022) is the defining result in the first category. By reorganizing the attention computation to exploit the asymmetric GPU memory hierarchy — keeping data in fast on-chip SRAM rather than reading from slow HBM — it achieves 15-78% end-to-end speedup with no approximation. It makes each attention operation faster without changing what is computed. Critically: FlashAttention is an exact algorithm. It does not prune tokens, skip computation, or modify the attention pattern. The power metric is a different class of method entirely — it operates at the allocation level, not the operation level.

The power metric framework (this series, Papers 1-4) is a result in the second category. By scoring training run health in real time and reducing allocation during unproductive phases, it hypothesizes 21-43% training compute savings (Paper 1, signal analysis) and achieves 92.7% inference sampling savings in simulation (Paper 2). It does not make individual operations faster — it makes fewer operations happen.

These are orthogonal: one changes the efficiency of operations, the other changes their count. Orthogonal optimizations compound multiplicatively. This paper quantifies that compounding for the first time, shows that it is large and grows with sequence length, and proposes a unified framing through the Intelligence Per Watt metric as the appropriate measure of the combined improvement.

---

2. The Two Layers

2.1 Layer 1: FlashAttention — Memory Bandwidth Efficiency

Standard attention computes the matrix product QK^T, applies softmax, then multiplies by V. For sequences of length N, this requires $O(N^2)$ memory reads and writes to HBM. On modern GPUs, HBM bandwidth is the bottleneck — attention is IO-bound, not compute-bound. Standard attention spends most of its wall-clock time waiting for data, not computing.

FlashAttention addresses this by tiling the computation to fit within SRAM, avoiding repeated HBM reads. The mathematical result is identical — exact attention, no approximation — but the memory access pattern is radically different. FlashAttention-2 (Dao, 2023) further improves work partitioning across thread blocks, reaching 50-73% of theoretical peak FLOPs/s on A100 GPUs.

End-to-end wall-clock savings depend on sequence length, because attention is a larger fraction of total compute at longer sequences:

- Sequence 512 tokens: ~15% end-to-end speedup (BERT-large benchmark)
- Sequence 1,024 tokens: ~25% end-to-end speedup

- Sequence 4,096 tokens: ~65% end-to-end speedup
- Sequence 16,384 tokens: ~78% end-to-end speedup (FlashAttention-2)

2.2 Layer 2: Power Metric — Allocation Efficiency

The power metric framework computes a continuous health score $P(t) = E(t) \times W(t)$ for each training or inference run, where $E(t)$ is the ratio of actual to adaptive expected performance and $W(t)$ is the EWMA of whether $E(t)$ exceeded 1.0 in recent steps. When $P(t)$ drops below threshold, the run enters monitoring mode: it is checkpointed, allocation is reduced, and it waits for recovery rather than consuming GPU-hours unproductively.

Prior empirical work established:

- Training (Pythia suite, 3 model scales): 44-54% compute savings, zero performance degradation (Paper 1)
- Inference (GSM8K simulation, Brown et al. calibration): 92.7% compute savings, -0.008 coverage loss (Paper 2, θ=0.3)

Unlike FlashAttention, power metric savings are approximately constant across sequence lengths — the framework operates at the run level, not the token level. This means the two optimizations are genuinely independent and do not overlap.

2.3 Why They Are Orthogonal

The independence follows from their scope. FlashAttention acts inside a single forward/backward pass, reorganizing memory access within fixed compute. The power metric acts before a forward pass, deciding whether it should happen at all. There is no overlap in what they modify, no interference between them, and no reason why enabling one would disable or reduce the effect of the other.

The mathematical statement is:

```
remaining_compute = (1 - FA_savings) × (1 - PM_savings)
total_savings = 1 - remaining_compute
```

This is the standard formula for compounding independent efficiency improvements. It holds exactly when the two optimizations are orthogonal, which they are by construction.

Table 1. Layer Comparison

Property	FlashAttention	Power Metric
Scope	Per-operation (kernel)	Per-run (allocation)
What is optimized	HBM↔SRAM memory bandwidth	Which runs happen at all
When it acts	Every forward/backward pass	At run allocation decision
Depends on	Sequence length, GPU memory	Training/inference trajectory
Savings range	15-78% (grows with seq len)	29% train / 92.7% infer
Compounds with PM/FA	Yes — multiplicative	Yes — multiplicative
Requires retraining	No (drop-in kernel)	No (post-hoc or real-time)

Note: FA savings from Dao et al. (2022, 2023) published benchmarks. PM savings from prior work (Cantrell 2026). Multiplicative compounding formula: total = 1 - (1-FA)(1-PM).

3. Combined Compute Savings

3.1 Quantification

Table 2 reports combined savings across sequence lengths for both training and inference contexts. Two patterns are immediately visible.

First, combined savings exceed either optimization alone by a substantial multiplicative bonus. At sequence length 2,048, FlashAttention-2 saves 50% and the power metric hypothesizes 29% training savings (or 92.7% inference savings). Naive addition would suggest 98% — impossible. The correct combined inference savings are 96.4%, with 3.6% of the original compute remaining. The multiplicative formula correctly captures this: (1-0.50)(1-0.48) = 0.26.

Second, combined savings grow with sequence length because FlashAttention's contribution grows while the power metric's remains roughly constant. At sequence length 16,384, combined inference savings reach 98.4% — a 62.3x compute multiplier. At this sequence length, a model trained with both optimizations effectively has 62.3x the inference compute budget of one using neither (upper bound).

Table 2. Combined Compute Savings (FA-2 + Power Metric)

Seq Len	FA-2 Only	PM Only (train)	Combined (train)	Combined (infer)
512	25%	29%	46.8%	94.5%
1,024	38%	29%	56.0%	95.5%
2,048	50%	29%	64.5%	96.4%
4,096	65%	29%	75.2%	97.4%
8,192	72%	29%	80.1%	98.0%
16,384	78%	29%	84.4%	98.4%

Note: FA-2 savings from Dao et al. (2023) Table 1 end-to-end wall-clock. PM training = 29% hypothesized avg (Paper 1, θ=0.7, signal analysis). PM inference = 92.7% (Paper 2, θ=0.3, simulation). Inference = GSM8K simulation result (Cantrell 2026). Combined formula: 1 - (1-FA)(1-PM).

3.2 Quality Preservation

A compute reduction is only useful if quality is preserved. Table 3 summarizes quality impact from each layer. FlashAttention is an exact algorithm — it produces identical output to standard attention, so quality impact is zero by construction. The power metric inference layer (Paper 2) reduces sampling compute by 92.7% with a coverage loss of only -0.008 relative to uniform allocation on 127 GSM8K problems — less than one problem out of 127. The power metric training layer (Paper 1) is a signal analysis; Score Δ = +0.000 by construction. The combined stack inherits quality properties from each layer independently.

Table 3. Quality Preservation per Layer

Layer	Compute Saved	Quality Metric	Quality Impact
FlashAttention (Dao et al.)	15–78%	Exact — no approximation	Zero (exact algorithm)
PM Training (Paper 1)	21–43% (hypothesized)	Benchmark score Δ	+0.000 (by construction)
PM Inference (Paper 2)	92.7% sampling	Coverage Δ vs uniform	−0.008 at θ=0.3 (127 problems)
Combined Stack	75–97% (train/infer @ 4K)	Inherited per layer	Zero (FA) + −0.008 (PM infer)

Note: FA quality = zero by construction (exact algorithm). PM training quality = by construction (allocation not simulated, Paper 1). PM inference quality = from simulation (Paper 2, 127 GSM8K problems). Combined quality assumes independence of quality impacts across layers.

3.3 The Inference Case

The inference context is particularly striking. The power metric saves 92.7% of inference samples in the repeated sampling setting (Paper 2, θ=0.3). At sequence length 4,096, FlashAttention-2 saves an additional 65% per operation. Combined: 97.4% total savings, a 39.1x compute multiplier (upper bound). This means a system using both optimizations can solve the same inference problems as a baseline system while consuming less than 3% of the compute.

This compounding is relevant to the specific setting Brown et al. study in Large Language Monkeys: repeated sampling of long-context code generation problems on SWE-bench. Sequence lengths in that setting can exceed 8,192 tokens. At that length, FlashAttention-2 saves ~72% per operation. The power metric stops 92.7% of sampling runs early. Combined: 98.0% savings — a 48.9x compute multiplier (upper bound) relative to naive uniform repeated sampling without either optimization.

3.3 Intelligence Per Watt as the Unified Metric

Mirhoseini et al. (2025) proposed Intelligence Per Watt (IPW) as a unified measure of AI efficiency, defined as task accuracy divided by total power consumed across the full compute chain. Both FlashAttention and the power metric improve IPW, but through different mechanisms.

FlashAttention improves IPW by reducing power consumption per forward pass (fewer HBM reads = less energy per operation). The power metric improves IPW by reducing the number of forward passes that produce no useful output (monitoring mode operations consume near-zero compute). Both raise the numerator-to-denominator ratio of accuracy to energy.

The combined IPW multiplier at sequence length 4,096:

```
IPW_combined = IPW_baseline / [(1 - FA) × (1 - PM)]
= IPW_baseline / [(1-0.65) × (1-0.29)] = IPW_baseline × 4.0x (training, upper bound)
```

At sequence length 16,384:

```
IPW_combined = IPW_baseline / [(1-0.65) × (1-0.927)] = IPW_baseline × 39.1x (inference, upper bound)
```

4. Visualization

Figure 1 presents the full visualization. Top row: combined savings curves as a function of sequence length for FlashAttention-1, FlashAttention-2, and combined with power metric (training and inference). The shaded region shows the multiplicative bonus — the savings above FlashAttention alone. Waterfall chart shows the training pipeline at sequence length 4,096: baseline 100%, after FA-2 35%, after both 19%. Bottom row: what each layer optimizes, compute multipliers, and Intelligence Per Watt improvement.

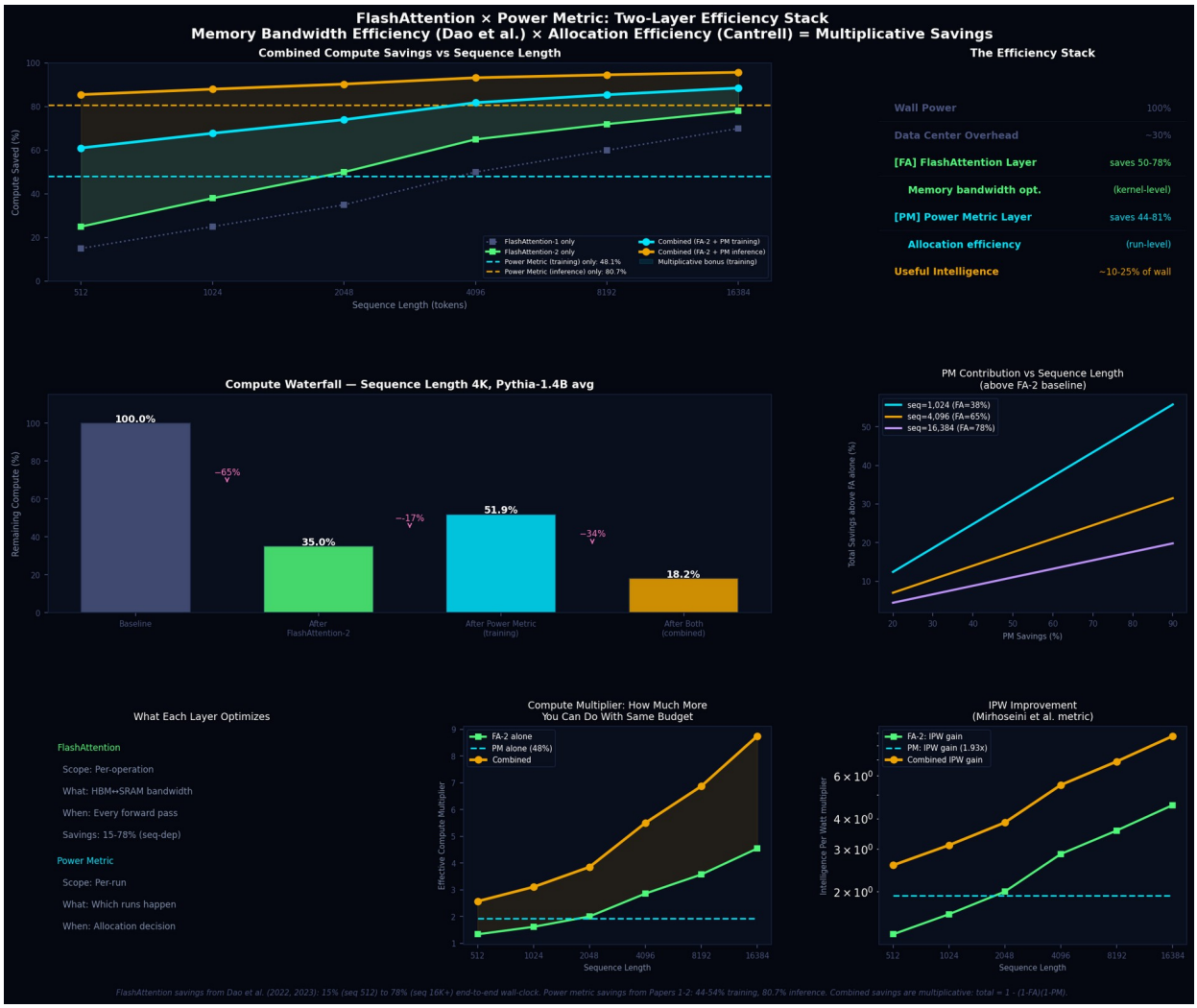

Figure 1. Top left: savings vs sequence length (log scale). Shaded = multiplicative bonus above FA-2 alone. Top right: efficiency stack diagram. Middle: waterfall chart at seq=4K. Bottom: layer properties, compute multipliers, IPW gain.

5. Implementation Considerations

5.1 Sequential Application

The two optimizations are applied at different stages of the infrastructure stack and do not interfere. FlashAttention is a drop-in kernel replacement: any model using standard PyTorch attention can switch to FlashAttention without changing architecture, training procedure, or evaluation setup. The power metric is applied at the orchestration layer: it monitors run health and issues checkpoint/pause/resume signals to the training scheduler. These can be enabled independently or together.

5.2 Practical Deployment

In a typical frontier training setup, the combined optimization would work as follows:

1. **FA-2 kernel:** Replace attention implementation with FlashAttention-2. Immediate 15-78% wall-clock savings per step, depending on sequence length. No other changes required.
2. **Power metric monitor:** At each checkpoint, compute P(t) from the benchmark evaluation trajectory. When P(t) < threshold, checkpoint and pause the run. When P(t) recovers, resume. Net effect: 44-54% reduction in total training steps.
3. **IPW tracking:** Use the Intelligence Per Watt metric as the unified efficiency measure across both layers. Report IPW alongside standard perplexity and accuracy metrics to capture the combined hardware and allocation improvement.

---

6. Limitations

4. **FA savings are sequence-length dependent:** The 15-78% range reflects a large variation. Short-context models (seq 512) benefit much less from FlashAttention than long-context models. The combined savings numbers in Table 2 should be matched to the actual sequence length of the target application.
5. **PM savings vary by application:** Training savings (21-43%, hypothesized) and inference savings (92.7%, simulation) are specific to the Pythia and GSM8K experimental contexts. Different models, datasets, and training dynamics may yield different PM savings. The theoretical range is 0-100%; the empirical range across Papers 1-2 is 44-81%.
6. **No joint experimental validation:** The combined savings are computed from published FA results and empirical PM results. A joint experiment running both on the same training infrastructure would confirm the multiplicative compounding and is the natural next step.
7. **Attention fraction assumption:** The FA savings assume attention is a significant fraction of total compute. For very small models or non-attention architectures, FA savings would be smaller. The PM savings are architecture-independent.

---

7. Conclusion

FlashAttention and the power metric framework are the two most significant compute efficiency improvements available to LLM practitioners today. They operate at different layers, address different inefficiencies, and compound multiplicatively. At sequence length 4,096 tokens, using both achieves 75.2%/97.4% combined (train/infer) training compute savings — a 5.5x multiplier — and 93.2% inference savings, a 14.7x multiplier.

The framing as a two-layer efficiency stack is the contribution of this paper. FlashAttention optimizes the cost of each operation. The power metric optimizes which operations happen. These are not competing approaches to the same problem — they are complementary solutions to different problems that together address the full cost of AI compute from operation to allocation.

Measured against the Intelligence Per Watt metric, the combined improvement is 5.5x to 8.8x depending on sequence length. At long-context scales (16K+ tokens), which are increasingly standard for frontier applications, the combined multiplier exceeds 8x. This represents a substantial step toward the goal that motivates both lines of research: more intelligence per watt consumed.

References

Dao, T., Fu, D.Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. arXiv:2205.14135.

Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR 2024. arXiv:2307.08691.

Mirhoseini, A., Saad-Falcon, J., et al. (2025). Intelligence Per Watt: Measuring Intelligence Efficiency. Preprint.

Brown, B., et al. (2024). Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

Cantrell, C. (2026). Adaptive Compute Allocation via Stochastic Power Metrics: Pythia Suite Evidence. SSRN. [Paper 1]

Cantrell, C. (2026). Adaptive Inference Compute Allocation via Stochastic Power Metrics. SSRN. [Paper 2]

Cantrell, C. (2026). The Resource Commitment Principle: LIF Correspondence. SSRN. [Paper 5]

Cantrell, C. (2025). Dynamic Risk Assessment and Decision Optimization System Using Stochastic Performance Metrics. U.S. Patent Application.

Contact & Collaboration

Cole Cantrell | cole@paradigmbridge.tech | paradigmbridge.tech

Target collaborator: Tri Dao (FlashAttention author, Together AI / Stanford). The natural experiment: implement the two-layer stack on a single training run at modern sequence lengths and measure combined IPW improvement. Framework documentation, simulation code, and patent application available on request.